

Random Class API

```
public class Random {  
    /** Creates a new random number generator.  Runs in  $\Theta(1)$  time. */  
    public Random() { ... }  
  
    /** Creates a new random number generator using a seed. If two instances of Random are created  
    with the same seed, they will generate and return identical sequences of numbers. Runs in  $\Theta(1)$   
    time. */  
    public Random(long seed) { ... }  
  
    /** Returns the next pseudorandom int from this random number generator's sequence. Runs in  $\Theta(1)$   
    time. */  
    public int nextInt() { ... }  
  
    /** Returns a pseudorandom int value between 0 (inclusive) and the specified bound (exclusive),  
    drawn from this random number generator's sequence. Runs in  $\Theta(1)$  time. */  
    public int nextInt(int bound) { ... }  
  
    /** Returns a pseudorandom int value between origin (inclusive) and the specified bound (exclusive),  
    drawn from this random number generator's sequence. Runs in  $\Theta(1)$  time. */  
    public int nextInt(int origin, int bound) { ... }  
}
```

Math Class API

```
public class Math {  
    /** Returns the value of the first argument raised to the power of the second argument.  
    * Runs in  $\Theta(1)$  time. */  
    public static double pow(double a, double b) { ... }  
  
    /** Returns the positive remainder when dividing x by y. Runs in  $\Theta(1)$  time. */  
    public static long floorMod(long x, long y) { ... }  
}
```

MinHeap Class API

```
public class MinHeap<E> {  
    /** Create a MinHeap<E>. Runs in  $\Theta(1)$  time. */  
    public MinHeap() { ... }  
  
    /** Inserts an element into the heap. Runs in  $O(\log N)$  time. */  
    public void insert(E element) { ... }  
  
    /** Deletes and returns the minimum element in the heap. If empty, return null.  
     * Runs in  $O(\log N)$  time. */  
    public E deleteMin() { ... }  
  
    /** Determines whether the heap is empty or not. Runs in  $\Theta(1)$  time. */  
    public boolean isEmpty() { ... }  
  
    /** Returns the size of the heap. Runs in  $\Theta(1)$  time. */  
    public int size() { ... }  
  
    /** Returns, but does not delete, the minimum element in the heap. This method is  
     * non-destructive. Runs in  $\Theta(1)$  time. */  
    public E findMin() { ... }  
}
```

MaxHeap Class API

```
public class MaxHeap<E> {  
    /** Create a MaxHeap<E>. Runs in  $\Theta(1)$  time. */  
    public MaxHeap() { ... }  
  
    /** Inserts an element into the heap. Runs in  $O(\log N)$  time. */  
    public void insert(E element) { ... }  
  
    /** Deletes and returns the maximum element in the heap. If empty, return null.  
     * Runs in  $O(\log N)$  time. */  
    public E deleteMax() { ... }  
  
    /** Determines whether the heap is empty or not. Runs in  $\Theta(1)$  time. */  
    public boolean isEmpty() { ... }  
  
    /** Returns the size of the heap. Runs in  $\Theta(1)$  time. */  
    public int size() { ... }  
  
    /** Returns, but does not delete, the maximum element in the heap. This method is  
     * non-destructive. Runs in  $\Theta(1)$  time. */  
    public E findMax() { ... }  
}
```

This is a piece of scratch paper.

